

**BABEȘ-BOLYAI UNIVERSITÄT CLUJ-NAPOCA
FAKULTÄT FÜR MATHEMATIK UND INFORMATIK
INFORMATIK IN DEUTSCHER SPRACHE**

BACHELORARBEIT

System zur Erkennung von ASL-Gesten basierend auf Convolutional Neural Networks

Betreuer

Lektor Dr. Cătălin Rusu

Assist. Doktorand Mădălina Dicu

*Eingereicht von
Matei Dana-Maria*

2026

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
INFORMATICĂ ÎN LIMBA GERMANĂ

LUCRARE DE LICENȚĂ

**Sistem pentru recunoașterea
gesturilor ASL bazat pe rețele
neuronale convoluționale**

Conducător științific

Lector Dr. Cătălin Rusu

Asist. Drd. Mădălina Dicu

*Absolvent
Matei Dana-Maria*

ABSTRACT

This thesis presents the design, development, and evaluation of a desktop application for real-time recognition of American Sign Language (ASL) fingerspelling gestures based on Convolutional Neural Networks (CNNs).

The work introduces the context of ASL gesture recognition and the theoretical foundations of CNNs, transfer learning, and the MobileNetV2 architecture. A combined dataset containing **103,379 images** from 24 static ASL letter classes was constructed by aggregating, cleaning, and standardizing seven publicly available Kaggle datasets. The data were then divided into training, validation, and test subsets using stratified sampling.

Two CNN-based approaches were developed and compared: a custom model trained from scratch and a MobileNetV2 model pre-trained on ImageNet and fine-tuned through transfer learning. The baseline model achieved a test accuracy of **98.17%**, while the MobileNetV2 model reached **99.59%**, demonstrating the effectiveness of transfer learning for this task.

The best-performing model was integrated into a desktop application, *ASL Recognition System*, which combines MediaPipe Hands for hand detection with the trained CNN for real-time gesture classification. The application also includes Firebase Authentication, a learning module for practicing the ASL alphabet, and an interactive game mode with text-to-speech feedback.

The obtained results show that the proposed system can recognize static ASL letters with very high accuracy, including visually similar gestures, while providing an accessible and practical tool for both ASL learning and communication support.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Ziele der Arbeit	1
1.2	Eigene Beiträge	2
1.3	Struktur der Arbeit	2
2	Theoretische Grundlagen und ASL-Kontext	4
2.1	Amerikanische Gebärdensprache (ASL)	4
2.2	Neuronale Netze und CNN	5
2.3	Datenaugmentation	6
2.4	Transfer Learning und MobileNetV2	7
2.4.1	Echtzeit-Handerkennung mit MediaPipe	8
2.5	Evaluationsmetriken	8
2.6	Verwandte Arbeiten	9
3	Datensätze und Vorverarbeitung	10
3.1	Datenquellen	10
3.2	Aufbau des kombinierten Datensatzes	10
3.3	Aufteilung in Teilmengen	11
3.4	Vorverarbeitung und Augmentierung	12
3.4.1	Normalisierung	12
3.4.2	Augmentierung	13
4	Architektur und Training der Modelle	14
4.1	Arbeitsumgebung	14
4.2	Custom-CNN-Modell	15
4.2.1	Architektur	15
4.2.2	Begründung der Designentscheidungen	16
4.2.3	Trainingskonfiguration	17
4.3	MobileNetV2-Modell mit Transfer Learning	17
4.3.1	Architektur des Klassifikationskopfes	17
4.3.2	Zweistufige Trainingsstrategie	17

4.4	Hyperparameter und Callback-Funktionen	18
4.4.1	Vergleich der Modelle	18
5	Experimente und Ergebnisse	19
5.1	Ergebnisse des CNN-Standardmodells	19
5.1.1	Trainingskurven	19
5.1.2	Leistung auf dem Testset	20
5.2	Ergebnisse des MobileNetV2-Modells	22
5.2.1	Trainingskurven	22
5.2.2	Leistung auf dem Testset	23
5.3	Vergleich der beiden Modelle	25
6	Implementierung der Anwendung	27
6.1	Gesamtarchitektur	27
6.2	Benutzerauthentifizierung	28
6.3	Pipeline der Echtzeit-Erkennung	29
6.4	Live-Erkennungsmodul	30
6.5	Modul zum Erlernen des ASL-Alphabets	31
6.6	Profilseite	32
7	Schlussfolgerungen	33
7.1	Zusammenfassung der Arbeit	33
7.2	Eigene Beiträge	33
7.3	Einschränkungen des Systems	34
7.4	Zukünftige Entwicklungsrichtungen	35
	Literaturverzeichnis	36

Abbildungsverzeichnis

4.1	Model Summary des Custom-CNN-Modells	16
5.1	Trainingskurven des CNN-Standardmodells (Genauigkeit und Verlust pro Epoche)	20
5.2	Klassifikationsbericht des CNN-Standardmodells auf dem Testset . .	20
5.3	Klassenweise Genauigkeit des CNN-Standardmodells auf dem Testset	21
5.4	Normalisierte Konfusionsmatrix des CNN-Standardmodells auf dem Testset	22
5.5	Trainingskurven des MobileNetV2-Modells (Phase 1 + Phase 2) . . .	23
5.6	Klassifikationsbericht des MobileNetV2-Modells auf dem Testset . .	24
5.7	Klassenweise Genauigkeit des MobileNetV2-Modells auf dem Testset	24
5.8	Normalisierte Konfusionsmatrix des MobileNetV2-Modells auf dem Testset	25
6.1	Login-Seite der Anwendung	29
6.2	Live-Erkennungsmodul mit Echtzeitvorhersage	30
6.3	Lernmodul zum Erlernen des ASL-Alphabets	31
6.4	Profilseite der Anwendung	32

Tabellenverzeichnis

3.1	Anzahl der Bilder pro Klasse im kombinierten Datensatz	11
3.2	Verteilung des Datensatzes auf Teilmengen	12
4.1	Trainingshyperparameter	18
5.1	Gesamtergebnisse des CNN-Standardmodells auf dem Testset	21
5.2	Buchstaben mit einer Genauigkeit unter 97% CNN-Standardmodell .	21
5.3	Gesamtergebnisse des MobileNetV2-Modells auf dem Testset	23
5.4	Vergleich zwischen CNN-Standardmodell und MobileNetV2	26

Kapitel 1

Einleitung

Kommunikation stellt eines der grundlegenden menschlichen Bedürfnisse dar. Für Menschen mit Hörbeeinträchtigungen ist die Gebärdensprache das wichtigste Mittel zur Ausdrucksweise und sozialen Interaktion. Dennoch bleibt die sprachliche Barriere zwischen Gebärdensprachnutzern und hörenden Personen eine bedeutende Herausforderung im Alltag, die den Zugang zu Bildung, Dienstleistungen und sozialen Möglichkeiten einschränkt. In diesem Kontext stellen automatische Gestenerkennungssysteme eine aktive Forschungsrichtung dar, mit dem realen Potenzial, diese Barrieren mithilfe von Technologie zu reduzieren.

Die automatische Erkennung von Gesten aus der *American Sign Language* (ASL) bleibt aus technischer Sicht ein schwieriges Problem. Die natürliche Variabilität der Gesten, Unterschiede zwischen Benutzern und unterschiedliche Lichtverhältnisse machen diese Aufgabe hochgradig komplex [1]. Vielfältige Hintergründe und die hohe visuelle Ähnlichkeit bestimmter Buchstaben des Alphabets sorgen dafür, dass diese Aufgabe selbst für moderne Systeme auf Basis tiefer neuronaler Netze weit davon entfernt ist, trivial zu sein [2]. In der Literatur wird immer wieder betont, că o clasificare robustă cere metodologii stricte de standardizare a datelor vizuale [3].

1.1 Ziele der Arbeit

Das Hauptziel dieser Arbeit besteht in der Entwicklung eines Systems zur automatischen Erkennung statischer Buchstaben des ASL-Alphabets aus Bildern, die über eine Webcam aufgenommen werden, basierend auf Convolutional Neural Networks. Im Rahmen dieses Hauptziels werden die folgenden spezifischen Ziele verfolgt:

- Untersuchung und Vergleich der Leistung zweier CNN-Architekturen zur Klassifikation der 24 statischen Buchstaben des ASL-Alphabets: ein von

Grund auf trainiertes CNN-Modell (*Baseline*) und ein Modell auf Basis von Transfer Learning (MobileNetV2 [4], vortrainiert auf ImageNet [5]);

- Aufbau eines robusten Trainingsdatensatzes durch Kombination mehrerer öffentlich verfügbarer Quellen auf der Kaggle-Plattform, gefolgt von manueller Bereinigung und Standardisierung;
- Entwicklung einer funktionalen Desktop-Anwendung, die das trainierte Modell für die Echtzeit-Erkennung von ASL-Gesten integriert, mit Firebase-Authentifizierung und einem interaktiven Lernmodul für das Alphabet.

1.2 Eigene Beiträge

Die originären Beiträge dieser Arbeit sind:

- Aufbau eines kombinierten Datensatzes von **103.379 ASL-Bildern**, gewonnen durch Aggregation, manuelle Bereinigung und Standardisierung von sieben öffentlichen Datensätzen, einschließlich Spiegelung von Bildern der linken Hand zur Vereinheitlichung auf die Konvention der rechten Hand;
- Entwurf, Training und vergleichende Evaluierung zweier CNN-Modelle , eines benutzerdefinierten von Grund auf trainierten Modells und MobileNetV2, angepasst durch Transfer Learning und Fine-Tuning in zwei Phasen , auf dem erstellten Datensatz;
- Entwicklung der Desktop-Anwendung *ASL Recognition System*, die die Echtzeit-Erkennungspipeline (MediaPipe Hands [6] + CNN-Modell + OpenCV), Firebase-Authentifizierung und ein interaktives Lernmodul für das ASL-Alphabet integriert.

1.3 Struktur der Arbeit

Die Arbeit ist in sieben Kapitel gegliedert, wie folgt.

Kapitel 2 präsentiert die notwendigen theoretischen Grundlagen: den sozialen und sprachlichen Kontext von ASL, technische Herausforderungen der Gestenerkennung, Prinzipien von Convolutional Neural Networks, das Konzept des Transfer Learnings, die Architektur von MobileNetV2 sowie verwendete Evaluierungsmetriken.

Kapitel 3 beschreibt den Aufbau des kombinierten Datensatzes , verwendete Quellen, Schritte der Bereinigung und Standardisierung, die Aufteilung in

Trainings-, Validierungs- und Testmengen sowie angewandte Vorverarbeitungs- und Augmentationstechniken.

Kapitel 4 detailliert die Architektur der beiden untersuchten Modelle, Designentscheidungen, Trainingsstrategie, verwendete Hyperparameter und die Arbeitsumgebung.

Kapitel 6 präsentiert die entwickelte Desktop-Anwendung: Gesamtarchitektur, Firebase-Authentifizierungsmodul, Echtzeit-Erkennungspipeline und das interaktive Lernmodul für das ASL-Alphabet.

Kapitel 5 stellt durchgeführte Experimente und erzielte Ergebnisse vor, einschließlich Vergleiche zwischen den beiden Modellen hinsichtlich Genauigkeit, Trainingskurven und Konfusionsmatrizen.

Kapitel 7 fasst die Beiträge der Arbeit zusammen, diskutiert Einschränkungen des Systems und schlägt zukünftige Entwicklungsrichtungen vor.

Kapitel 2

Theoretische Grundlagen und ASL-Kontext

2.1 Amerikanische Gebärdensprache (ASL)

Die Amerikanische Gebärdensprache (*American Sign Language*, ASL) ist eine vollständige visuell-gestische Sprache, unabhängig von der englischen Sprache, mit eigenen grammatikalischen Regeln und syntaktischer Struktur. ASL ist eine der am weitesten verbreiteten Gebärdensprachen der Welt [7], und aus kultureller Perspektive betrachtet die Gemeinschaft der Menschen mit Hörbeeinträchtigungen Taubheit nicht als Behinderung, sondern als sprachliche und kulturelle Differenz. Dennoch bleibt die Kommunikation zwischen ASL-Nutzern und hörenden Personen schwierig, und automatische Gestenerkennungssysteme haben das Potenzial, diese Barriere erheblich zu reduzieren.

Das ASL-Fingerspelling-Alphabet (*fingerspelling*) ermöglicht die Darstellung jedes Buchstabens des englischen Alphabets durch eine spezifische Handkonfiguration und wird für Eigennamen oder Begriffe ohne eigenes Zeichen verwendet. Aus Sicht der computerbasierten Verarbeitung können die 26 Buchstaben in zwei Kategorien unterteilt werden: **statische Gesten** (24 Buchstaben, A-Z mit Ausnahme von J und Z), die durch ein einzelnes Bild dargestellt werden können, und **dynamische Gesten** (J und Z), die eine Bewegungsbahn erfordern und die Analyse einer zeitlichen Bildsequenz benötigen. Diese Arbeit behandelt ausschließlich die **24 statischen Buchstaben**.

Visuelle Herausforderungen. Die automatische Erkennung von ASL-Gesten umfasst mehrere technische Schwierigkeiten. Bestimmte Buchstabengruppen weisen sehr ähnliche Handkonfigurationen auf; beispielsweise unterscheiden sich die Buchstaben M, N, S und T nur durch subtile Variationen der Fingerposition und des Daumens [1]. Lichtvariationen, typisch bei Webcams, beeinflussen das

visuelle Erscheinungsbild derselben Geste erheblich, und die Vielfalt der Hintergründe in realen Szenarien kann die Leistung von unter kontrollierten Bedingungen trainierten Modellen verringern. Zusätzlich sind ASL-Gesten naturgemäß asymmetrisch: eine horizontale Spiegelung führt entweder zu einer ungültigen Geste oder zur Darstellung eines anderen Buchstabens, ein wichtiger Aspekt bei der Datenaugmentation.

2.2 Neuronale Netze und CNN

Maschinelles Lernen (*Machine Learning*, ML) ist ein Teilbereich der künstlichen Intelligenz, der Algorithmen entwickelt, die in der Lage sind, Muster in Daten zu erkennen, ohne explizit programmiert zu sein. Diese Arbeit folgt dem Paradigma des **überwachten Lernens** (*supervised learning*): Das Modell wird anhand von Beispielen mit bekannten Antworten trainiert und lernt, Eingaben den korrekten Klassen zuzuordnen, indem es den Vorhersagefehler minimiert. Die behandelte Aufgabe ist eine **Multi-Klassen-Klassifikation**, bei der jedes Bild einer von mehreren festen Kategorien zugeordnet wird, und die Ausgabe des Modells ein Wahrscheinlichkeitsvektor mit Summe 1 ist.

Die grundlegende Einheit neuronaler Netze ist das **künstliche Neuron**: Es empfängt Eingabewerte, kombiniert diese durch eine gewichtete Summe, addiert einen Bias-Term und wendet eine Aktivierungsfunktion an, um einen einzelnen Ausgabewert zu erzeugen. Solche mathematischen Strukturen bilden das Fundament moderner tiefer Architekturen [8]. Um komplexe nichtlineare Beziehungen zu modellieren, werden Neuronen in **mehrschichtige Feedforward-Netze** (*Multi-Layer Perceptron*, MLP) organisiert, mit einer Eingabeschicht, einer oder mehreren verborgenen Schichten und einer Ausgabeschicht, wobei die Information in eine Richtung propagiert. Das Training basiert auf dem **Backpropagation**-Algorithmus, der den Beitrag jedes Parameters zum Gesamtfehler berechnet und ihn iterativ anpasst, um diesen zu minimieren. Aktivierungsfunktionen führen Nichtlinearität in das Netz ein: **ReLU** (*Rectified Linear Unit*) reduziert das Problem des *vanishing gradient*, bei dem Gradienten in tiefen Schichten extrem klein werden und das Training blockieren [9], während **Softmax** in der Ausgabeschicht die Rohwerte des Netzes in eine geordnete Wahrscheinlichkeitsverteilung transformiert.

Vollständig verbundene (*fully connected*) Netze, die direkt auf Bilder angewendet werden, erzeugen eine untragbar große Anzahl von Parametern und ignorieren die räumliche Struktur des Bildes sowie die Beziehungen zwischen benachbarten Pixeln, die Kanten, Texturen und Formen bilden. **Convolutional Neural Networks** (CNNs) lösen beide Probleme durch Parameterteilung und die Nutzung lokaler Pixelbeziehungen [9]. Die **Convolution-Schicht** wendet kleine Filter (*Kernels*) an,

die über das Bild gleiten und *Feature Maps* erzeugen, welche das Vorhandensein bestimmter Muster kodieren. Filter in frühen Schichten erkennen einfache Merkmale wie Kanten und Kontraste, während tiefere Schichten komplexe Formen und Strukturen identifizieren und so eine Hierarchie visueller Repräsentationen bilden. **MaxPooling** reduziert die räumliche Dimension der Feature Maps, indem es den Maximalwert in jedem Fenster beibehält, wodurch der Rechenaufwand sinkt und eine gewisse Translationsinvarianz entsteht. Nach den Convolution-Blöcken transformiert die **Flatten**-Operation die dreidimensionalen Repräsentationen in einen eindimensionalen Vektor, der anschließend an **Dense**-Schichten (*fully connected*) zur eigentlichen Klassifikation weitergegeben wird.

Regularisierung und Optimierung. *Overfitting*, also die Tendenz eines Modells, Trainingsdaten zu memorisieren statt zu generalisieren, wird durch mehrere Techniken verhindert. **Dropout** deaktiviert zufällig einen Teil der Neuronen während des Trainings und zwingt das Netzwerk, robustere Repräsentationen zu lernen [10]. **L2-Regularisierung** bestraft große Parameterwerte, indem sie einen Term proportional zur Summe der Quadrate dieser Werte zur Verlustfunktion hinzufügt. **Batch Normalization** normalisiert die Aktivierungen innerhalb jedes Mini-Batches, stabilisiert das Training und beschleunigt die Konvergenz [11]. Für das Training wird die Verlustfunktion *categorical cross-entropy* verwendet, und der **Adam**-Optimierer passt die Lernrate für jeden Parameter individuell an, wodurch eine schnellere und stabilere Konvergenz erreicht wird [12]. **ReduceLROnPlateau** reduziert automatisch die Lernrate, wenn die überwachte Metrik stagniert, **Early Stopping** stoppt das Training, wenn auf dem Validierungsset kein Fortschritt mehr erzielt wird, und **ModelCheckpoint** speichert automatisch das beste Modell während des Trainings.

2.3 Datenaugmentation

Das Training eines Bildklassifikationsmodells erfordert große und vielfältige Datensätze, um eine gute Generalisierungsfähigkeit zu erreichen. Wenn die verfügbaren Daten begrenzt oder nicht ausreichend vielfältig sind, besteht die Gefahr, dass das Modell die Trainingsdaten auswendig lernt und auf neuen Daten schlecht performt. **Datenaugmentation** (*data augmentation*) ist eine Regularisierungstechnik, die die Vielfalt des Trainingssatzes künstlich erweitert, indem zufällige Transformationen auf vorhandene Bilder angewendet werden, ohne zusätzliche Daten zu sammeln [13]. In der Praxis schützt dieses Verfahren das System vor Überanpassung, indem geometrische und pixelweise Variationen generiert werden.

Die in dieser Arbeit verwendeten Transformationen umfassen moderate Rotationen zur Simulation unterschiedlicher Handorientierungen; horizontale und vertikale Verschiebungen zur Reduzierung der Abhängigkeit von der exakten Hand-

position im Bild; Zoom zur Simulation unterschiedlicher Kameradistanzen; Helligkeitsschwankungen zur Nachbildung unterschiedlicher Lichtverhältnisse; sowie Scherung (*shear*) zur Modellierung leicht geneigter Perspektiven. Ein wichtiger ASL-spezifischer Aspekt ist, dass horizontales Spiegeln (*horizontal flip*) bewusst deaktiviert wurde, da Gebärden asymmetrisch sind: eine Spiegelung kann entweder eine ungültige Geste oder einen anderen Buchstaben darstellen und somit Rauschen in die Trainingsdaten einführen. Die Augmentation wird ausschließlich auf dem Trainingsset angewendet; Validierungs- und Testsets verwenden die Originalbilder ohne Transformationen, um eine korrekte und reproduzierbare Bewertung der Modellleistung sicherzustellen.

2.4 Transfer Learning und MobileNetV2

Das Training eines CNN von Grund auf erfordert große Datensätze und erhebliche Rechenressourcen. **Transfer Learning** [14] adressiert dieses Problem, indem Wissen eines bereits auf einer anderen Aufgabe trainierten Modells wiederverwendet wird: grundlegende visuelle Merkmale wie Kanten, Texturen und Formen sind in vielen visuellen Aufgaben gemeinsam und können mit minimalen Anpassungen übertragen werden. **ImageNet** [5], mit über 1,2 Millionen Bildern in 1.000 Kategorien, ist der historische Meilenstein für das Vortraining von Modellen der künstlichen Bildverarbeitung [15]. Der Prozess der Anpassung eines vortrainierten Modells an eine neue Aufgabe wird **Fine-Tuning** genannt und umfasst typischerweise zwei Phasen: Zunächst werden nur die neu hinzugelegten Schichten oberhalb der eingefrorenen Basis trainiert, anschließend werden die oberen Schichten der Basis freigeschaltet und mit einer reduzierten Lernrate trainiert, um die bereits gelernten Repräsentationen nicht zu zerstören.

MobileNetV2 [4] ist eine CNN-Architektur, die von Google für effiziente Inferenz auf ressourcenbeschränkten Geräten entwickelt wurde. Ihre Effizienz basiert auf zwei Innovationen: Depthwise-Separable-Convolutions, die eine Standard-Convolution in zwei separate Operationen zerlegen und dadurch die Rechenkosten erheblich reduzieren, sowie Inverted-Residual-Blöcke, inspiriert von ResNet [16], die direkte Verbindungen zwischen Ein- und Ausgang hinzufügen, um den Gradientenfluss in tiefen Netzen zu erleichtern. Trotz ihrer geringen Rechenkosten erzielt MobileNetV2 wettbewerbsfähige Ergebnisse im Vergleich zu deutlich größeren Architekturen, was sie zu einer geeigneten Wahl für Echtzeitanwendungen macht.

2.4.1 Echtzeit-Handerkennung mit MediaPipe

Die Erkennung von Gesten aus einem Videostream in Echtzeit erfordert nicht nur die Klassifikation der Geste, sondern auch die vorherige Lokalisierung der Hand im Bild. Ohne diesen Schritt würde das Klassifikationsmodell das gesamte Webcam-Bild inklusive Hintergrund erhalten, was die Genauigkeit und Robustheit der Vorhersagen erheblich reduzieren würde. **MediaPipe Hands** [6] ist ein von Google entwickeltes Framework, das die Echtzeit-Handerkennung und -verfolgung direkt auf dem Gerät (*on-device*) ermöglicht, ohne einen externen Server zu benötigen. Generell bietet dieses Ökosystem hocheffiziente Werkzeuge für Cross-Platform Multimedia-Pipelines [17]. Die Lösung erkennt die Hand im Bild und schätzt die Position von 21 Schlüsselpunkten (*landmarks*) der Hand mit geringer Latenz, geeignet für interaktive Anwendungen. In dieser Arbeit wird MediaPipe Hands verwendet, um eine Region of Interest (*bounding box*) um die erkannte Hand zu bestimmen, die anschließend zugeschnitten, auf die erwartete Eingangsgröße des Modells skaliert und zur Klassifikation weitergeleitet wird.

2.5 Evaluationsmetriken

Die Leistung eines Klassifikationsmodells wird durch mehrere komplementäre Metriken bewertet. **Accuracy**, also der Anteil korrekt klassifizierter Beispiele, ist die wichtigste Vergleichsmetrik. Die **Loss-Funktion** (*categorical cross-entropy*) misst die Differenz zwischen der vorhergesagten Wahrscheinlichkeitsverteilung und dem korrekten Label; ihre Entwicklung während des Trainings signalisiert das Auftreten von *Overfitting*. **Precision** gibt an, welcher Anteil der Vorhersagen einer Klasse korrekt ist, **Recall** zeigt, welcher Anteil der tatsächlichen Beispiele einer Klasse korrekt erkannt wird, und der **F1-Score**, das harmonische Mittel beider Größen, bietet eine ausgewogene Bewertung pro Klasse, insbesondere wenn beide Aspekte wichtig sind.

Die **Confusion Matrix** (*Konfusionsmatrix*) zeigt die Verteilung der Vorhersagen für jedes Paar aus tatsächlicher und vorhergesagter Klasse: die Hauptdiagonale enthält die korrekten Vorhersagen, während die übrigen Elemente Klassifikationsfehler darstellen. Die **normalisierte** Variante (Anteile pro tatsächlicher Klasse) erleichtert den Vergleich zwischen Klassen unterschiedlicher Größe und hilft dabei, häufig verwechselte Buchstabenpaare zu identifizieren, wodurch ein tieferes Verständnis der Modellgrenzen als durch die reine Gesamtgenauigkeit ermöglicht wird.

2.6 Verwandte Arbeiten

Die automatische Erkennung von ASL-Gesten mittels Convolutional Neural Networks wurde in mehreren aktuellen Arbeiten untersucht, die die Machbarkeit dieses Ansatzes zeigen und Referenzpunkte für die Bewertung des in dieser Arbeit entwickelten Systems bieten.

Taskiran et al. [18] schlagen ein CNN-basiertes System zur Erkennung statischer ASL-Gesten vor und zeigen, dass Convolutional-Architekturen gute Ergebnisse auch bei großen Datensätzen mit hoher Benutzervariabilität erzielen. Pisharady und Saerbeck [1] erweitern diesen Ansatz durch Fine-Tuning auf vortrainierten Modellen und erreichen bessere Leistungen als beim Training von Grund auf, insbesondere in Szenarien mit begrenzten Daten, was die Effektivität von Transfer Learning für Echtzeit-Gestenerkennung bestätigt. Benitez-Garcia et al. [2] führen einen systematischen Vergleich mehrerer CNN-Architekturen zur Klassifikation statischer RGB-Gesten durch und zeigen, dass sowohl die Wahl der Architektur als auch die Trainingsstrategie die Endleistung signifikant beeinflussen.

Im Vergleich zu diesen Arbeiten zeichnet sich die vorliegende Arbeit durch die Kombination eines Forschungsanteils, nämlich den Vergleich eines von Grund auf trainierten CNN-Modells mit einem Transfer-Learning-Modell (MobileNetV2), mit der Entwicklung einer funktionalen Desktop-Anwendung aus, die das trainierte Modell für die Echtzeit-Erkennung integriert und ein interaktives Lernmodul für das ASL-Alphabet bereitstellt.

Kapitel 3

Datensätze und Vorverarbeitung

3.1 Datenquellen

Das Training eines ASL-Gestenerkennungsmodells erfordert einen ausreichend großen und vielfältigen Datensatz, um die natürliche Variabilität der Gesten abzudecken, Unterschiede zwischen Nutzern, Lichtverhältnisse und Hintergründe. Da kein öffentlich verfügbarer Datensatz als individuell ausreichend repräsentativ betrachtet wurde, wurden die Trainingsdaten durch die Kombination von sieben öffentlich verfügbaren Datensätzen aus der Kaggle-Plattform gewonnen: den *ASL American Sign Language Alphabet Dataset* [19], den *American Sign Language Dataset* [20], den *ASL Alphabet* [21], den *ASL Alphabet Test* [22], den *ASL RGB-Depth Fingerspelling: Spelling It Out* [23], den *Significant ASL Sign Language Alphabet Dataset* [24] und den *ASL Hand Alphabet Dataset* [25]. Alle Datensätze enthalten RGB-Bilder der Hand bei der Ausführung von Buchstaben des ASL-Fingerspelling-Alphabets, aufgenommen unter variierenden Lichtbedingungen und mit unterschiedlichen Hintergründen.

3.2 Aufbau des kombinierten Datensatzes

Der Aufbau des finalen Datensatzes umfasste mehrere Schritte. Zunächst wurden die aus den sieben Quellen stammenden Bilder zusammengeführt und nach Klassen organisiert (ein Ordner pro ASL-Buchstabe). Da einige Datensätze Bilder mit der linken Hand und andere mit der rechten Hand enthalten, war eine Standardisierung auf eine einheitliche Konvention erforderlich. Die Anwendung richtet sich an rechtshändige Nutzer, daher wurden alle Bilder auf die Darstellung mit der **rechten Hand** vereinheitlicht: Bilder, die als mit der linken Hand aufgenommen identifiziert wurden, wurden manuell erkannt und horizontal gespiegelt (*horizontal flip*), um die äquivalente Darstellung der rechten Hand zu erhalten.

Zweitens wurde eine **manuelle Auswahl** der Bilder im kombinierten Datensatz durchgeführt. Bilder von schlechter Qualität, mit uneindeutigen Gesten oder solche, die nicht der Darstellungskonvention entsprachen, wurden entfernt, sodass nur klare und repräsentative Beispiele für jede Klasse erhalten blieben. In der Literatur zur Gestenerkennung wird betont, dass die manuelle Bereinigung von Rauschen die Konvergenzgeschwindigkeit neuronaler Netze maßgeblich beschleunigt [26]. Dieser Schritt stellte die Qualität und Konsistenz des finalen Datensatzes sicher.

Der finale Datensatz enthält **103.379 Bilder**, verteilt auf **24 Klassen**, entsprechend den statischen Buchstaben des ASL-Alphabets (A-Z, mit Ausnahme von J und Z). Die Verteilung ist nicht perfekt ausgeglichen: Die Anzahl der Bilder pro Klasse variiert zwischen **3.003** (Buchstabe D) und **4.988** (Buchstabe W), was die unterschiedliche Verfügbarkeit von Beispielen in den ursprünglichen Quellen widerspiegelt. Die vollständige Klassenverteilung ist in Tabelle 3.1 dargestellt.

Tabelle 3.1: Anzahl der Bilder pro Klasse im kombinierten Datensatz

Buchstabe	Bildanzahl	Buchstabe	Bildanzahl
A	4.355	N	3.901
B	4.588	O	4.330
C	3.437	P	3.620
D	3.003	Q	3.888
E	3.461	R	4.922
F	4.616	S	4.916
G	4.363	T	4.129
H	3.491	U	4.947
I	4.703	V	4.624
K	4.866	W	4.988
L	4.871	X	4.729
M	3.867	Y	4.764
Gesamt: 103.379 Bilder			

3.3 Aufteilung in Teilmengen

Der kombinierte Datensatz wurde in drei disjunkte Teilmengen aufgeteilt, unter Verwendung eines speziellen Python-Skripts, das eine zufällige, stratifizierte Aufteilung auf Klassenebene durchführte, wobei sichergestellt wurde, dass die Proportionen für jeden Buchstaben unabhängig eingehalten werden. Um die Reproduzierbarkeit der Ergebnisse zu gewährleisten, wurden alle Zufallsoperationen durch einen festen Seed-Wert fixiert (`SEED = 42`).

- **Trainingsset (train)**, 80% der Bilder, verwendet zur Anpassung der Modellgewichte;
- **Validierungsset (val)**, 10% der Bilder, verwendet zur Überwachung der Leistung während des Trainings und für Callback-Entscheidungen (Early Stopping, ReduceLROnPlateau, ModelCheckpoint);
- **Testset (test)**, 10% der Bilder, ausschließlich zur finalen Bewertung der Modelle nach Abschluss des Trainings.

Das Ergebnis der Aufteilung ist in Tabelle 3.2 dargestellt:

Tabelle 3.2: Verteilung des Datensatzes auf Teilmengen

Teilmenge	Bildanzahl	Proportion
Training	82.692	80%
Validierung	10.327	10%
Test	10.360	10%
Gesamt	103.379	100%

Die strikte Trennung des Testsets vom Trainingsprozess ist entscheidend, um eine objektive Bewertung der Modellleistung zu erhalten. Die Daten des Testsets wurden zu keinem Zeitpunkt während des Trainings oder der Hyperparameteranpassung verwendet.

3.4 Vorverarbeitung und Augmentierung

Die Vorverarbeitung und Augmentierung der Daten wurde in Google Colab mithilfe der Klasse `ImageDataGenerator` aus TensorFlow/Keras implementiert. Es wurden zwei separate Generatoren definiert: einer für das Trainingsset, der sowohl Normalisierung als auch zufällige Augmentierung anwendet, und einer für die Validierungs- und Testsets, der ausschließlich Normalisierung anwendet.

3.4.1 Normalisierung

Alle Bilder, unabhängig vom Subset, werden auf 128×128 Pixel **skaliert** und durch Division der Pixelwerte durch 255 (`rescale=1.0/255`) **normalisiert**, wodurch die Werte aus dem Bereich $[0, 255]$ in den Bereich $[0, 1]$ überführt werden. Die Normalisierung beschleunigt die Konvergenz des Trainings und sorgt für numerische Stabilität.

3.4.2 Augmentierung

Die Datenaugmentation wird ausschließlich auf das Trainingsset angewendet und besteht aus zufälligen Transformationen der Bilder während des Trainings, wodurch künstliche Variationen der vorhandenen Beispiele erzeugt werden. Ziel der Augmentierung ist es, die scheinbare Vielfalt der Trainingsdaten zu erhöhen und *Overfitting* zu reduzieren. Die angewandten Transformationen sind:

- **Zufällige Rotation** bis zu ± 20 (`rotation_range=20`), simuliert Variationen des Handwinkels zur Kamera;
- **Horizontale und vertikale Verschiebung** von bis zu 15% der Bildgröße (`width_shift_range=0.15`, `height_shift_range=0.15`), simuliert variable Positionierung der Hand im Bild;
- **Zufälliger Zoom** bis zu 15% (`zoom_range=0.15`), simuliert Variationen der Entfernung zur Kamera;
- **Zufällige Helligkeitsvariation** im Bereich `[0.6, 1.4]` (`brightness_range=[0.6, 1.4]`), simuliert unterschiedliche Lichtbedingungen, von relativ dunkel bis relativ hell;
- **Schertransformation** (*shear*) bis zu 0.1 Radiant (`shear_range=0.1`), führt leichte geometrische Verzerrungen ein, um die Robustheit des Modells zu erhöhen;
- **Horizontales Flip deaktiviert** (`horizontal_flip=False`), horizontales Spiegeln wird nicht angewendet, da ASL-Gesten asymmetrisch sind; eine Spiegelung würde entweder eine ungültige Geste oder einen anderen Buchstaben darstellen.

Die durch geometrische Transformationen entstehenden Pixel (z. B. leere Bereiche nach Rotation oder Verschiebung) werden mittels des nächstgelegenen Pixelwerts gefüllt (`fill_mode="nearest"`).

Kapitel 4

Architektur und Training der Modelle

Dieses Kapitel beschreibt detailliert die beiden im Rahmen der Arbeit entwickelten und verglichenen CNN-Modelle: ein von Grund auf trainiertes CNN-Modell, das als Referenzlinie (*Baseline*) dient, und ein Modell basierend auf der MobileNetV2-Architektur, das durch Transfer Learning für die ASL-Gestenklassifikation angepasst wurde. Beide Modelle erhalten als Eingabe RGB-Bilder der Größe $128 \times 128 \times 3$ Pixel und geben eine Wahrscheinlichkeitsverteilung über die 24 Klassen der statischen Buchstaben des ASL-Alphabets aus.

4.1 Arbeitsumgebung

Das Training beider Modelle wurde in **Google Colaboratory** (Colab) durchgeführt, einer Cloud-Plattform, die kostenlosen Zugriff auf GPU-Hardwarebeschleuniger bietet. Die Verwendung der GPU reduzierte die Trainingszeit erheblich im Vergleich zur CPU-Ausführung. Zur weiteren Beschleunigung wurde Mixed Precision (*mixed precision float16/float32*) aktiviert, das 16-Bit-Darstellungen für Berechnungen und 32-Bit für Gewichte verwendet und dadurch den Speicherverbrauch sowie die Zeit pro Epoche um etwa 30-40% reduziert.

Die Modelle wurden in **Python** unter Verwendung des **TensorFlow 2.x**-Frameworks mit der High-Level-API **Keras** implementiert. Der Datensatz wurde in **Google Drive** gespeichert und bei jeder Trainingssitzung in die Colab-Umgebung eingebunden; Modelle und Trainingsverläufe wurden nach jeder Epoche in Drive gespeichert, wodurch ein Fortsetzen des Trainings bei Sitzungsablauf ermöglicht wurde. Die Reproduzierbarkeit der Experimente wurde durch das Fixieren des Zufalls-Seed-Werts auf 42 für TensorFlow, NumPy, Python sowie die Umgebungsvariable `PYTHONHASHSEED` gewährleistet.

4.2 Custom-CNN-Modell

4.2.1 Architektur

Das Custom-CNN-Modell wurde von Grund auf als Referenzlinie (*Baseline*) entworfen, mit dem Ziel, ein minimales Leistungsniveau festzulegen, anhand dessen das Transfer-Learning-Modell bewertet werden kann. Die Architektur folgt der klassischen CNN-Struktur: Convolution-Blöcke zur Merkmalsextraktion, gefolgt von Dense-Schichten zur Klassifikation.

Das Netzwerk besteht aus **drei aufeinanderfolgenden Convolution-Blöcken**, die jeweils eine Conv2D-Schicht mit Filtergröße 3×3 , gefolgt von BatchNormalization und MaxPooling 2×2 enthalten. Die Anzahl der Filter steigt progressiv an 32, 64 und 128 um dem Netzwerk zu ermöglichen, eine Hierarchie von Repräsentationen zu lernen: von einfachen Mustern (Kanten, Kontraste) im ersten Block bis hin zu komplexen Handkonfigurationen im dritten Block, die spezifisch für ASL-Buchstaben sind. Alle Convolution-Schichten verwenden die ReLU-Aktivierungsfunktion.

Nach den drei Convolution-Blöcken werden die Feature-Maps durch eine **Flatten**-Schicht (resultierend in einem Vektor von 25.088 Elementen) abgeflacht und an eine **Dense**-Schicht mit 256 Neuronen übergeben, mit ReLU-Aktivierung und L2-Regularisierung mit Koeffizient 10^{-4} . Anschließend folgt eine **Dropout**-Schicht mit einer Rate von 0.5 zur Vermeidung von *Overfitting*, und schließlich die Ausgabeschicht **Dense** mit 24 Neuronen und Softmax-Aktivierung, die die Wahrscheinlichkeitsverteilung über die 24 ASL-Klassen erzeugt.

Die vollständige Modellarchitektur ist in Abbildung 4.1 dargestellt.

Das Modell besitzt insgesamt **6.523.096 Parameter**. Davon sind **6.522.648 trainierbare Parameter** und **448 nicht trainierbare Parameter**, die aus den BatchNormalization-Schichten stammen. Zusätzlich enthält das Modell **13.045.302 Optimizer-Parameter**, die vom Adam-Optimierer intern für den Trainingsprozess gespeichert werden.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
batch_normalization (BatchNormalization)	(None, 126, 126, 32)	128
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 61, 61, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 28, 28, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 256)	6,422,784
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 24)	6,168

Total params: 19,568,398 (74.65 MB)
 Trainable params: 6,522,648 (24.88 MB)
 Non-trainable params: 448 (1.75 KB)
 Optimizer params: 13,045,302 (49.76 MB)

Abbildung 4.1: Model Summary des Custom-CNN-Modells

4.2.2 Begründung der Designentscheidungen

Die Wahl von drei Convolution-Blöcken mit steigender Filteranzahl ($32 \rightarrow 64 \rightarrow 128$) stellt einen Kompromiss zwischen Repräsentationsfähigkeit und dem Risiko von *Overfitting* bei einem Datensatz mittlerer Größe dar. Die Reduzierung der räumlichen Dimensionen folgt erprobten Praktiken zur automatischen Merkmalsextraktion [26]. Eine geringere Anzahl von Blöcken wäre nicht ausreichend, um die komplexen Handkonfigurationen zu erfassen, während eine größere Anzahl die Modellgröße unnötig erhöhen und das Risiko des Auswendiglernens der Trainingsdaten verstärken würde.

BatchNormalization nach jeder Convolution-Schicht stabilisiert die Verteilung der Aktivierungen während des Trainings, ermöglicht höhere Lernraten und beschleunigt die Konvergenz. L2-Regularisierung auf der Dense-Schicht sowie Dropout(0.5) wirken komplementär gegen *Overfitting*: L2 bestraft große Gewichte, während Dropout das Netzwerk zwingt, nicht von einzelnen Neuronen abhängig

zu sein.

4.2.3 Trainingskonfiguration

Das Custom-CNN-Modell verwendet eine einzige Trainingsphase. Die Kompilierung erfolgte mit dem **Adam**-Optimierer mit einer initialen Lernrate $lr = 10^{-4}$ und der Verlustfunktion *categorical cross-entropy*. Die maximale Anzahl an Epochen wurde auf 40 gesetzt; das Training lief über alle **40 Epochen**, ohne dass EarlyStopping eingriff, was darauf hindeutet, dass das Modell während des gesamten Trainings Fortschritte machte, wobei am Ende automatisch die Gewichte der besten Epoche wiederhergestellt wurden.

4.3 MobileNetV2-Modell mit Transfer Learning

4.3.1 Architektur des Klassifikationskopfes

Das MobileNetV2-Modell wurde mit auf ImageNet vortrainierten Gewichten initialisiert (`weights="imagenet"`), wobei die ursprüngliche Klassifikationsschicht entfernt wurde (`include_top=False`), während die Convolution-Basis als Merkmalsextraktor erhalten blieb. Der ImageNet-Datensatz selbst dient in der Fachwelt als anerkannter Benchmark für allgemeine visuelle Repräsentationen [27]. Die Eingabe hat die Dimension $128 \times 128 \times 3$ und ist damit kompatibel mit der Architektur. Über der vortrainierten Basis wurde ein benutzerdefinierter Klassifikationskopf für die 24 ASL-Klassen hinzugefügt, bestehend aus:

- **GlobalAveragePooling2D**
- **Dense(512)** mit L2-Regularisierung(10^{-4}), BatchNormalization, ReLU und Dropout(0.4)
- **Dense(256)** mit L2-Regularisierung(10^{-4}), BatchNormalization, ReLU und Dropout(0.3)
- **Dense(24)** mit Softmax

Die Verwendung von GlobalAveragePooling2D reduziert die Anzahl der Parameter erheblich und verbessert die Generalisierung.

4.3.2 Zweistufige Trainingsstrategie

Die Anpassung von MobileNetV2 erfolgte durch eine zweistufige Trainingsstrategie.

Phase 1: Nur der Klassifikationskopf wird trainiert (Backbone eingefroren), Lernrate 10^{-3} , 20 Epochen.

Phase 2: Teilweises Fine-Tuning der oberen Schichten mit Lernrate 10^{-5} für ca. 30 Epochen.

4.4 Hyperparameter und Callback-Funktionen

Tabelle 4.1 zeigt die Hyperparameter beider Modelle.

Tabelle 4.1: Trainingshyperparameter

Hyperparameter	Custom CNN	MobileNetV2
Bildgröße	128×128	128×128
Batch Size	32	32
Epochen	40	20 + 30
Optimizer	Adam	Adam
Loss	Categorical cross-entropy	Categorical cross-entropy
Dropout	0.5	0.4 / 0.3
L2	10^{-4}	10^{-4}

4.4.1 Vergleich der Modelle

Die beiden Modelle unterscheiden sich wesentlich darin, dass das Custom CNN vollständig trainiert wird, während MobileNetV2 vortrainierte ImageNet-Features nutzt und nur angepasst wird. Dadurch erreicht MobileNetV2 bessere Generalisierung bei geringerem Trainingsaufwand.

Kapitel 5

Experimente und Ergebnisse

Dieses Kapitel präsentiert die Ergebnisse, die im Zuge des Trainings und der Evaluierung der beiden im Rahmen dieser Arbeit entwickelten CNN-Modelle erzielt wurden. Die Evaluierung erfolgte auf dem Testset, das aus 10.360 Bildern besteht, die zu keinem Zeitpunkt während des Trainings oder der Hyperparameter-Optimierung verwendet wurden. Analysiert werden die Trainingskurven, die globale Genauigkeit, die Klassifikationsberichte pro Klasse sowie die Konfusionsmatrizen, gefolgt von einem direkten Vergleich der beiden Modelle.

5.1 Ergebnisse des CNN-Standardmodells

5.1.1 Trainingskurven

Das CNN-Standardmodell lief alle 40 konfigurierten Epochen, ohne dass EarlyStopping eingriff, was darauf hinweist, dass das Modell während des gesamten Trainings weitere Fortschritte machte. Die Genauigkeits- und Verlustkurven sind in Abbildung 5.1 dargestellt.

Ein bemerkenswerter Aspekt der Kurven ist, dass die Validierungsgenauigkeit während des gesamten Trainings konstant über der Trainingsgenauigkeit liegt. Dieses scheinbar kontraintuitive Verhalten lässt sich durch die starke Datenaugmentation erklären, die ausschließlich auf das Trainingsset angewendet wurde: Rotationen, Translationen, Zoom und Helligkeitsschwankungen machen die Trainingsbilder deutlich schwieriger als die unveränderten Validierungsbilder. Daher stellt diese Differenz kein *Overfitting* dar, sondern spiegelt die unterschiedliche Schwierigkeit der Datensätze wider. Sowohl Trainings- als auch Validierungsgenauigkeit konvergieren am Ende zu hohen Werten, was eine gute Generalisierungsfähigkeit bestätigt.

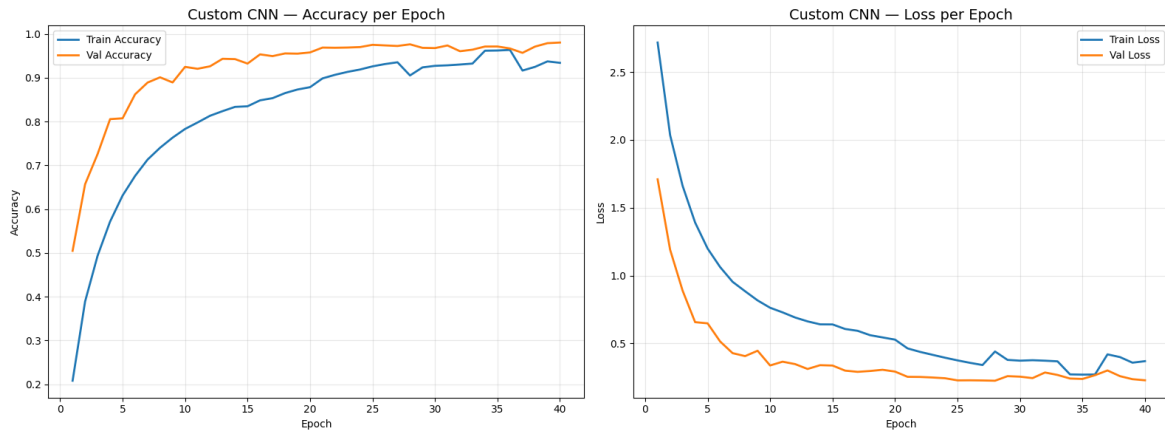


Abbildung 5.1: Trainingskurven des CNN-Standardmodells (Genauigkeit und Verlust pro Epoche)

5.1.2 Leistung auf dem Testset

Auf dem Testset erreichte das CNN-Standardmodell eine **Genauigkeit von 98,17%** und einen Verlust von 0,2253. Der vollständige Klassifikationsbericht ist in Abbildung 5.2 dargestellt, die aggregierten Ergebnisse sind in Tabelle 5.1 zusammengefasst.

Class	Precision	Recall	F1-Score	Support
A	0.9882	0.9610	0.9744	436
B	0.9638	0.9848	0.9742	460
C	0.9855	0.9884	0.9870	345
D	0.9932	0.9767	0.9849	301
E	0.9793	0.9539	0.9664	347
F	0.9705	0.9935	0.9819	463
G	0.9977	0.9886	0.9931	437
H	0.9831	0.9971	0.9901	350
I	0.9790	0.9894	0.9842	471
K	0.9898	0.9939	0.9918	488
L	0.9918	0.9918	0.9918	488
M	0.9529	0.9897	0.9709	388

Class	Precision	Recall	F1-Score	Support
N	0.9894	0.9565	0.9727	391
O	0.9953	0.9838	0.9895	433
P	1.0000	0.9945	0.9972	362
Q	1.0000	0.9974	0.9987	390
R	0.9772	0.9574	0.9672	493
S	0.9775	0.9696	0.9735	493
T	0.9952	1.0000	0.9976	414
U	0.9569	0.9839	0.9702	496
V	0.9699	0.9741	0.9720	463
W	0.9741	0.9780	0.9760	500
X	0.9745	0.9684	0.9714	474
Y	0.9916	0.9895	0.9906	477

Metric	Precision	Recall	F1-Score	Support
Accuracy			0.9817	10360
Macro Avg	0.9824	0.9817	0.9820	10360
Weighted Avg	0.9818	0.9817	0.9817	10360

Abbildung 5.2: Klassifikationsbericht des CNN-Standardmodells auf dem Testset

Tabelle 5.1: Gesamtergebnisse des CNN-Standardmodells auf dem Testset

Metrik	Wert
Genauigkeit (accuracy)	98,17%
Verlust (loss)	0,2253
Macro avg Precision	98,24%
Macro avg Recall	98,17%
Macro avg F1-Score	98,20%

Auf Klassenebene erreichen die meisten Buchstaben eine Genauigkeit von über 97%. Die sechs Buchstaben mit einer Leistung unterhalb dieses Schwellenwerts sind in Tabelle 5.2 dargestellt:

Tabelle 5.2: Buchstaben mit einer Genauigkeit unter 97% CNN-Standardmodell

Buchstabe	Genauigkeit
E	95,39%
N	95,65%
R	95,74%
A	96,10%
X	96,84%
S	96,96%

Die klassenweise Genauigkeit ist in Abbildung 5.3 visualisiert. Die Buchstaben E und N weisen die schwächsten Ergebnisse auf, was visuell erklärbar ist: E, N, M und S besitzen sehr ähnliche Handkonfigurationen, die sich nur durch subtile Unterschiede in der Position des Daumens und der gekrümmten Finger unterscheiden.

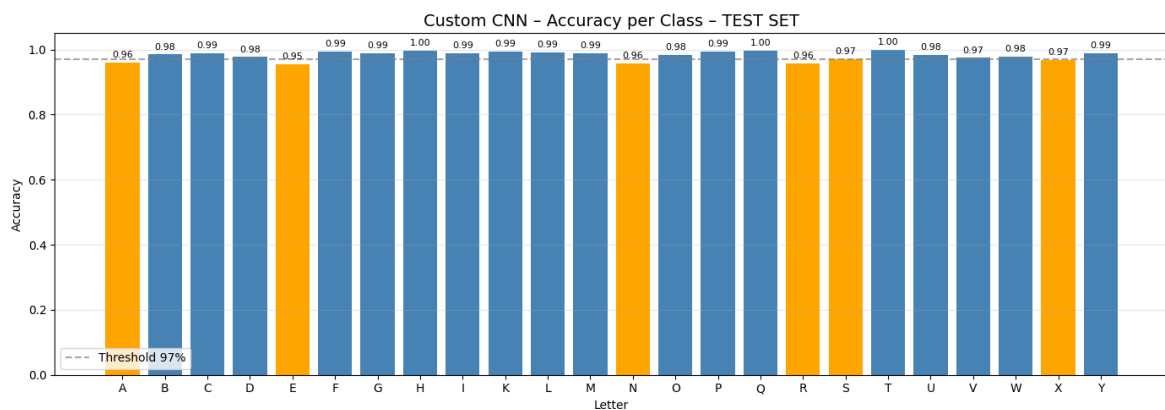


Abbildung 5.3: Klassenweise Genauigkeit des CNN-Standardmodells auf dem Testset

Dies wird auch durch die Konfusionsmatrix bestätigt (Abbildung 5.4), wobei der häufigste Einzelfehler die Verwechslung $N \rightarrow M$ (16 Fälle in absoluten Werten) ist.

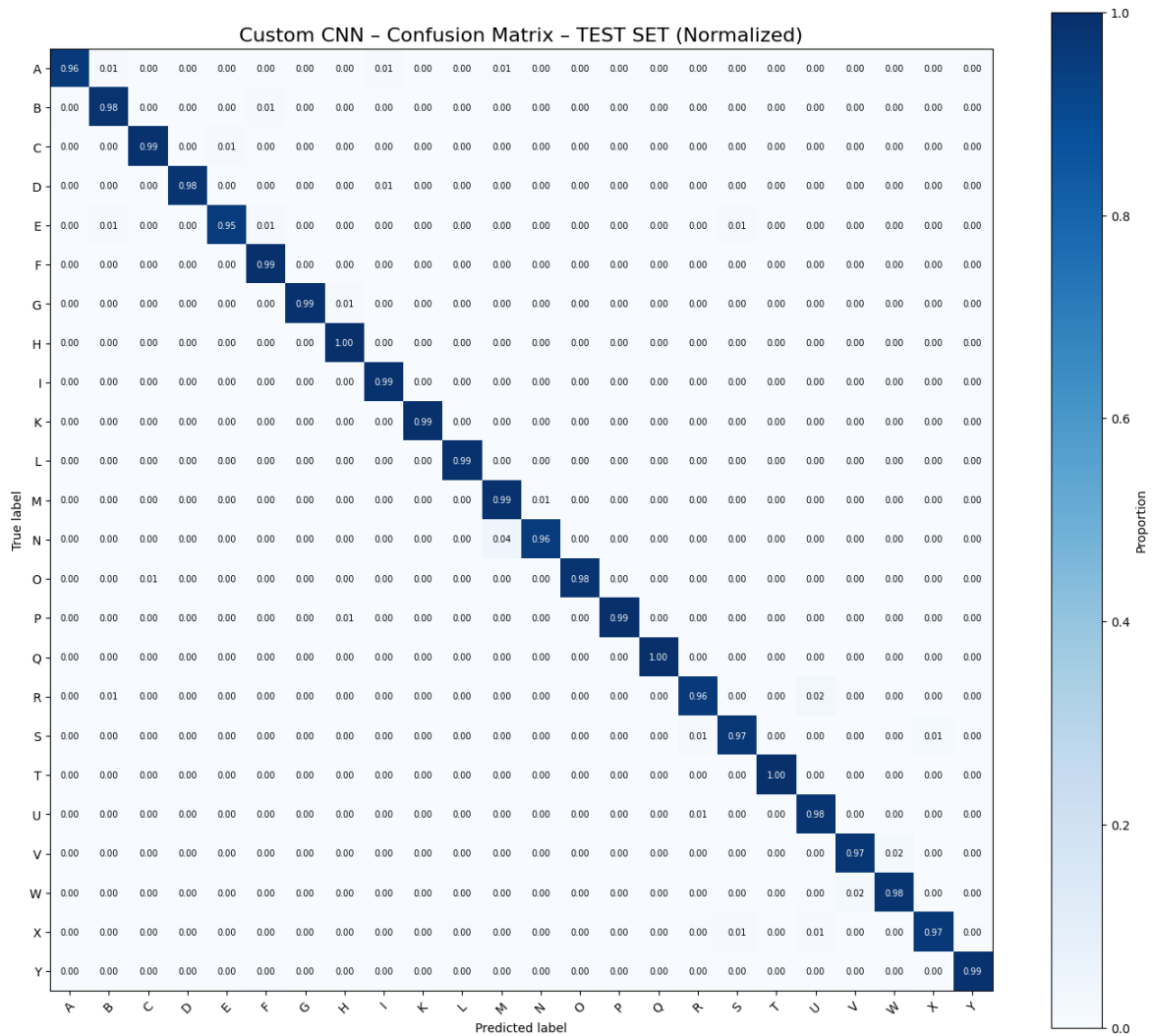


Abbildung 5.4: Normalisierte Konfusionsmatrix des CNN-Standardmodells auf dem Testset

5.2 Ergebnisse des MobileNetV2-Modells

5.2.1 Trainingskurven

Das Training von MobileNetV2 umfasste zwei Phasen mit insgesamt etwa 50 Epochen, dargestellt in Abbildung 5.5. Der Übergang zwischen den Phasen (Epoche 20) verursacht einen deutlichen Spike in beiden Kurven: Die Genauigkeit fällt abrupt, während der Verlust stark ansteigt. Dieses Verhalten ist typisch und erwartet beim Fine-Tuning, da das Entfrieren der Basisschichten und die neue Konfiguration die zuvor gelernten Repräsentationen vorübergehend destabilisieren. Das Modell erholt sich jedoch schnell und konvergiert zu deutlich besseren Werten.

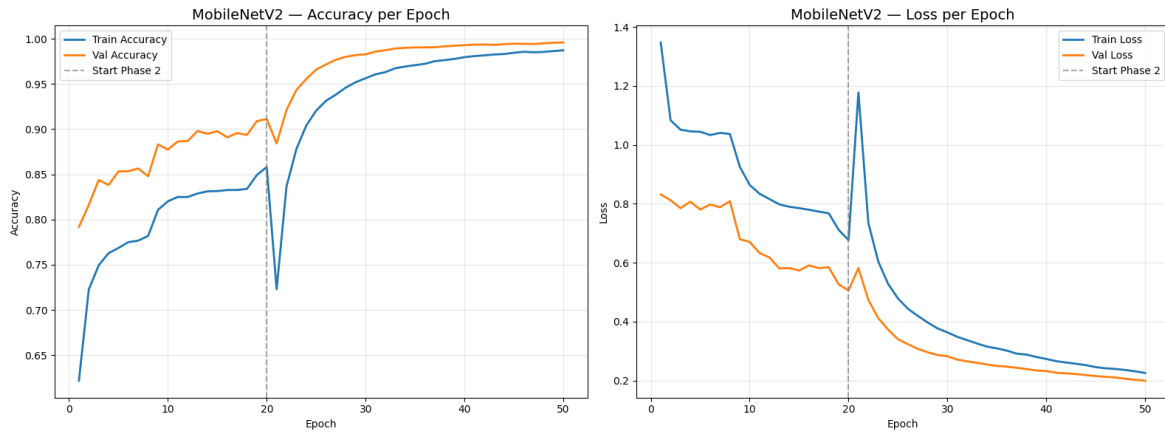


Abbildung 5.5: Trainingskurven des MobileNetV2-Modells (Phase 1 + Phase 2)

Im Gegensatz zum CNN-Standardmodell bleibt die Trainingsgenauigkeit in Phase 1 leicht unter der Validierungsgenauigkeit, übertrifft diese jedoch in Phase 2 zunehmend, während sich die oberen freigegebenen Schichten auf ASL-Gesten spezialisieren. Beide Kurven konvergieren am Ende des Trainings gegen sehr hohe Werte nahe 1.0.

5.2.2 Leistung auf dem Testset

Auf dem Testset erreichte das MobileNetV2-Modell eine **Genauigkeit von 99,59%** und einen Verlust von 0,1968. Die aggregierten Ergebnisse in Tabelle 5.3 dargestellt, der Klassifikationsbericht ist in Abbildung 5.6.

Tabelle 5.3: Gesamtergebnisse des MobileNetV2-Modells auf dem Testset

Metrik	Wert
Genauigkeit (accuracy)	99,59%
Verlust (loss)	0,1968
Macro avg Precision	99,60%
Macro avg Recall	99,62%
Macro avg F1-Score	99,61%

Class	Precision	Recall	F1-Score	Support	Class	Precision	Recall	F1-Score	Support
A	0.9977	0.9954	0.9966	436	N	0.9974	0.9923	0.9949	391
B	0.9978	1.0000	0.9989	460	O	1.0000	1.0000	1.0000	433
C	1.0000	0.9971	0.9985	345	P	1.0000	1.0000	1.0000	362
D	0.9869	1.0000	0.9934	301	Q	1.0000	1.0000	1.0000	390
E	0.9971	1.0000	0.9986	347	R	0.9939	0.9858	0.9898	493
F	1.0000	0.9957	0.9978	463	S	0.9939	0.9980	0.9960	493
G	0.9977	0.9977	0.9977	437	T	0.9976	1.0000	0.9988	414
H	0.9971	0.9971	0.9971	350	U	0.9859	0.9859	0.9859	496
I	0.9894	0.9936	0.9915	471	V	0.9913	0.9870	0.9892	463
K	0.9959	1.0000	0.9980	488	W	0.9920	0.9980	0.9950	500
L	1.0000	0.9939	0.9969	488	X	1.0000	0.9979	0.9989	474
M	0.9923	0.9974	0.9949	388	Y	1.0000	0.9958	0.9979	477

Metric	Precision	Recall	F1-Score	Support
Accuracy			0.9959	10360
Macro Avg	0.9960	0.9962	0.9961	10360
Weighted Avg	0.9960	0.9959	0.9959	10360

Abbildung 5.6: Klassifikationsbericht des MobileNetV2-Modells auf dem Testset

Ein bemerkenswertes Ergebnis des MobileNetV2-Modells ist, dass **alle 24 Klassen** eine Genauigkeit von mindestens 99% erreichen, wobei keine einzige Klasse unter diesen Schwellenwert fällt. Die klassenweise Genauigkeit ist in Abbildung 5.7 dargestellt.

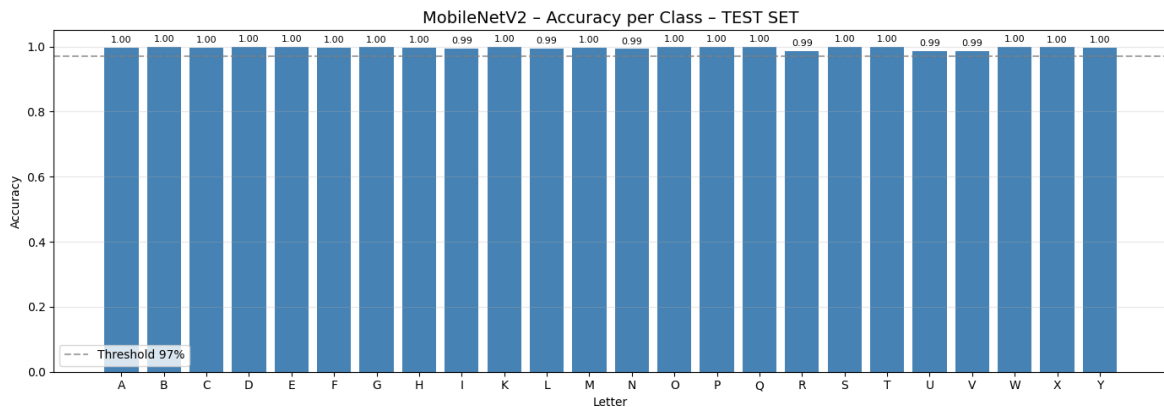


Abbildung 5.7: Klassenweise Genauigkeit des MobileNetV2-Modells auf dem Testset

Die normalisierte Konfusionsmatrix (Abbildung 5.8) ist nahezu perfekt diagonal, mit isolierten Fehlern von maximal 1% pro Klasse, was die überlegene Diskriminationsfähigkeit des Modells bestätigt.

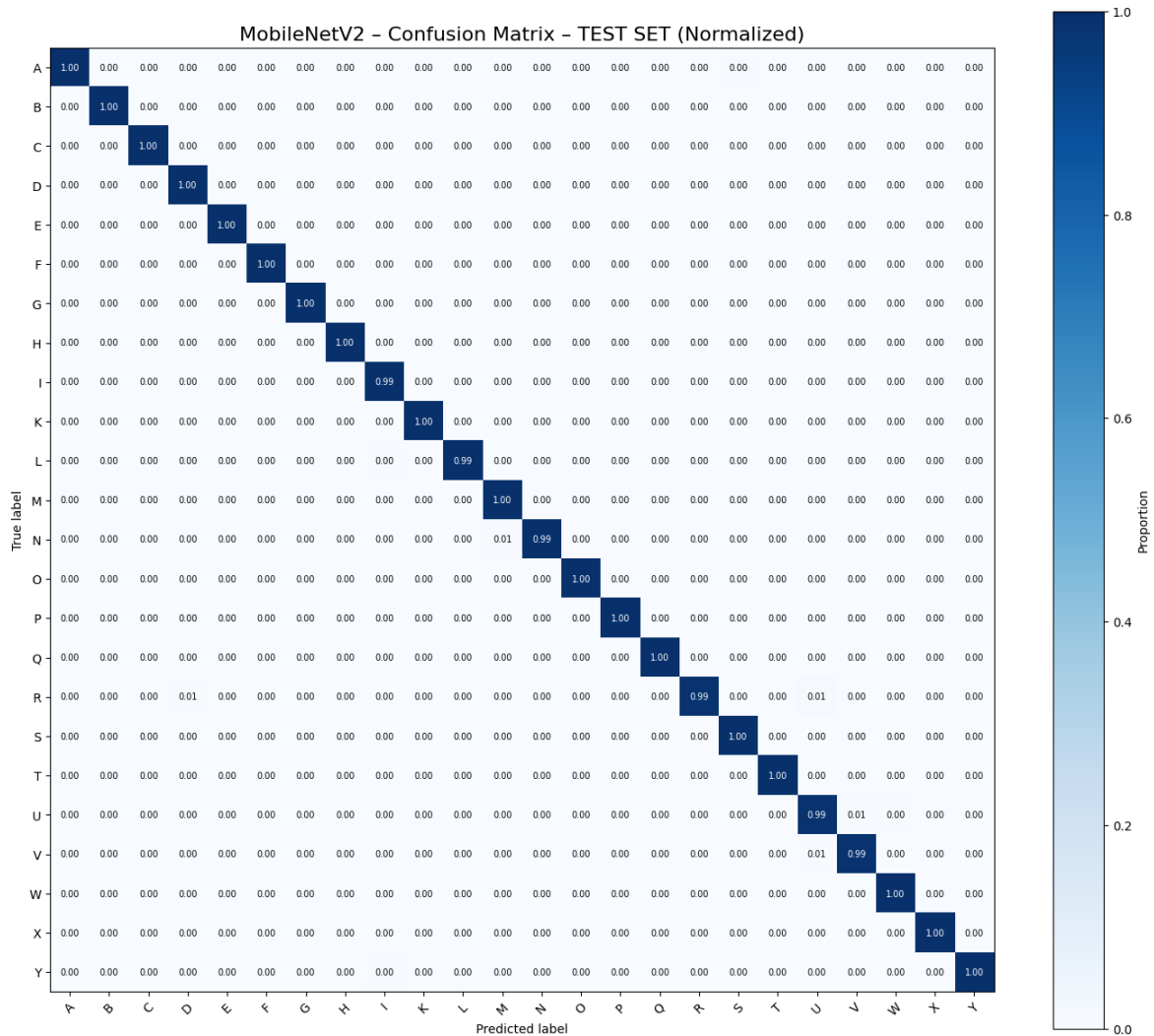


Abbildung 5.8: Normalisierte Konfusionsmatrix des MobileNetV2-Modells auf dem Testset

5.3 Vergleich der beiden Modelle

Tabelle 5.4 zeigt einen direkten Vergleich der beiden Modelle anhand der wichtigsten Evaluationsmetriken:

Tabelle 5.4: Vergleich zwischen CNN-Standardmodell und MobileNetV2

Metrik	CNN-Standardmodell	MobileNetV2
Testgenauigkeit	98,17%	99,59%
Testverlust	0,2253	0,1968
Macro avg F1-Score	98,20%	99,61%
Gesamtparameter	6.523.096	≈3.500.000
Klassen unter 97%	6	0
Trainings-Epochen	40	≈50

Der Unterschied von **1,42 Prozentpunkten** in der Genauigkeit ist im Kontext der 24-Klassen-Klassifikation signifikant. MobileNetV2 erreicht bessere Leistung mit etwa der Hälfte der Parameter des CNN-Standardmodells und zeigt damit deutlich den Vorteil des Transfer Learning: Die auf ImageNet erlernten visuellen Repräsentationen ermöglichen eine präzisere Merkmalsextraktion aus ASL-Daten, selbst bei geringerer Modellkapazität [27]. Zudem zeigt der Einsatz von gemischter Präzision (*mixed precision*), wie Berechnungen auf modernen Grafikprozessoren ohne Genauigkeitsverlust optimiert werden können [28].

Die problematischen Buchstaben des CNN-Standardmodells (E, N, S, M) gehören zu den visuell ähnlichsten Zeichen des ASL-Alphabets und unterscheiden sich nur durch subtile Fingerpositionen. MobileNetV2 kann diese Klassen nahezu fehlerfrei trennen, was durch die nahezu perfekt diagonale Konfusionsmatrix bestätigt wird.

Basierend auf diesen Ergebnissen wurde das **MobileNetV2-Modell** für die Integration in die Desktop-Anwendung ausgewählt, da es das beste Gleichgewicht zwischen Genauigkeit, Effizienz und Robustheit pro Klasse bietet.

Kapitel 6

Implementierung der Anwendung

Dieses Kapitel beschreibt die Architektur und Implementierung der im Rahmen dieser Arbeit entwickelten Desktop-Anwendung mit dem Namen *ASL Recognition System*. Die Anwendung integriert das für die Echtzeit-Erkennung von ASL-Gesten trainierte MobileNetV2-Modell und bietet dem Benutzer zwei Hauptmodule: ein Live-Erkennungsmodul und ein interaktives Modul zum Erlernen des ASL-Alphabets.

6.1 Gesamtarchitektur

Die Anwendung folgt einer **Client-Server-Architektur** mit klarer Trennung zwischen Frontend und Backend. Das **Backend** ist in Python mit dem Framework **FastAPI** implementiert und läuft lokal auf dem Rechner des Benutzers, manuell über das Terminal mit dem Server `uvicorn` gestartet. FastAPI glänzt hierbei vor allem durch seine asynchrone Verarbeitungsgeschwindigkeit und automatische Dokumentation [29]. Das **Frontend** ist in reinem HTML, CSS und JavaScript implementiert und läuft im Browser (Chrome, Firefox usw.), wobei es über HTTP-Anfragen mit dem Backend kommuniziert.

Die Projektstruktur ist in zwei Hauptmodule organisiert:

- **backend/** enthält den FastAPI-Server, die Authentifizierungslogik, die Vorhersage-Pipeline und das ML-Modell:
 - `app/api/` - REST-Routen (`predict.py`, `users.py`);
 - `app/core/` - Firebase-Authentifizierungs-Utilities (`auth_utils.py`, `firebase_config.py`);
 - `app/ml/` - Inferenz-Pipeline (`asl_model.py`, `hand_detector.py`, `preprocess.py`);

- `models/` - trainiertes Modell (`mobilenetv2_model_final.keras`) und Klassendatei (`classes.json`);
- **frontend/** - enthält die Benutzeroberfläche:
 - HTML-Dateien für jede Seite der Anwendung;
 - CSS-Dateien für das Styling;
 - JavaScript-Dateien für die Logik der Benutzeroberfläche und die Kommunikation mit dem Backend.

6.2 Benutzerauthentifizierung

Die Verwaltung der Benutzerkonten erfolgt über **Firebase Authentication**, einen Cloud-Dienst von Google. Der Benutzer kann sich mit E-Mail-Adresse und Passwort registrieren und anmelden. Die tokenbasierte Absicherung schützt Endpunkte vor unbefugten Zugriffen über standardisierte Sicherheitsmechanismen [30]. Bei erfolgreicher Authentifizierung gibt Firebase ein **JWT-Token** (*JSON Web Token*) zurück, das lokal im Browser (`localStorage`) gespeichert und bei jeder weiteren HTTP-Anfrage im Header `Authorization: Bearer <token>` mitgesendet wird.

Auf der Backend-Seite überprüft jeder geschützte Endpunkt die Gültigkeit des JWT-Tokens über das Modul `auth_utils.py`, das das Firebase Admin SDK verwendet. Wenn das Token fehlt oder ungültig ist, wird die Anfrage mit dem HTTP-Code 401 abgelehnt. Dieser Ansatz stellt sicher, dass nur authentifizierte Benutzer auf die Funktionen der Anwendung, einschließlich der Modellvorhersagen, zugreifen können.

Benutzerdaten (Maximalpunktzahl, gelernte Buchstaben) werden in **Firestore**, einer NoSQL-Cloud-Datenbank, gespeichert und vom Backend über das Firebase Admin SDK abgerufen. Die Anwendung enthält außerdem Hilfeseiten zur Passwortwiederherstellung und Kontolöschung.

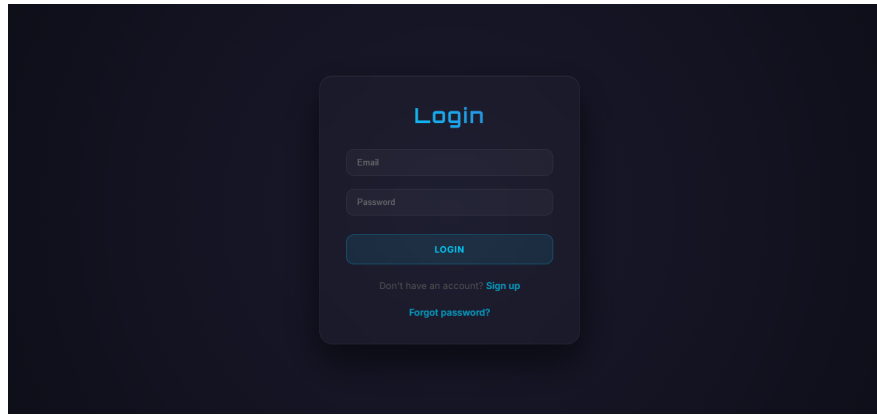


Abbildung 6.1: Login-Seite der Anwendung

6.3 Pipeline der Echtzeit-Erkennung

Die Pipeline der Echtzeit-Erkennung stellt die zentrale Komponente der Anwendung dar und beinhaltet die Zusammenarbeit zwischen Frontend und Backend in jedem verarbeiteten Videoframe. Der vollständige Ablauf ist wie folgt:

1. **Videoaufnahme.** Das Frontend greift über die `getUserMedia`-API des Browsers auf die Webcam des Benutzers zu. Die Videoframes werden kontinuierlich erfasst und vor der Verarbeitung horizontal gespiegelt, um der natürlichen Perspektive des Benutzers zu entsprechen.
2. **Übertragung des Frames.** Bei jeder Iteration der Vorhersageschleife (`requestAnimationFrame`) kodiert das Frontend den aktuellen Frame als JPEG-Bild und sendet ihn über eine HTTP-Anfrage im Format `multipart/form-data` an den Backend-Endpunkt `POST /predict`, zusammen mit dem Authentifizierungstoken.
3. **Handerkennung.** Das Backend empfängt den Frame und übergibt ihn an das Modul `hand_detector.py`, das **MediaPipe Hands** verwendet, um die Hand zu erkennen und die *Bounding Box* zu berechnen. Wenn keine Hand erkannt wird, erfolgt keine Vorhersage.
4. **Vorverarbeitung.** Die durch die *Bounding Box* definierte Region of Interest wird aus dem Frame ausgeschnitten und vom Modul `preprocess.py` vorverarbeitet: auf 128×128 Pixel skaliert und durch Division der Pixelwerte durch 255 normalisiert, identisch zur Vorverarbeitung während des Trainings.
5. **Modell-Inferenz.** Das vorverarbeitete Bild wird an das MobileNetV2-Modell (`asl_model.py`) übergeben, das einen Wahrscheinlichkeitsvektor über die 24

ASL-Klassen erzeugt. Die Klasse mit der höchsten Wahrscheinlichkeit stellt den vorhergesagten Buchstaben dar, und der entsprechende Wert ist der *Confidence-Score*.

6. **Annotation und Rückgabe des Ergebnisses.** Das Backend zeichnet die erkannte *Bounding Box* und den vorhergesagten Buchstaben auf den ursprünglichen Frame, kodiert das annotierte Bild im Base64-Format und gibt es als JSON-Antwort an das Frontend zurück, einschließlich des vorhergesagten Buchstabens, des *Confidence-Scores* und des annotierten Bildes.
7. **Stabilisierung der Vorhersage durch Majority Voting.** Das Frontend speichert einen zirkulären Puffer der letzten 7 Vorhersagen sowie der zugehörigen *Confidence-Scores*. Der angezeigte Buchstabe wird durch **Majority Voting** bestimmt: Der am häufigsten im Puffer vorkommende Buchstabe wird als stabile Vorhersage betrachtet. Solche Abstimmungsalgorithmen sind in fehlertoleranten Entscheidungssystemen weit verbreitet [31]. Der angezeigte *Confidence-Score* entspricht dem Durchschnitt der Werte im Puffer. Dieser Ansatz eliminiert Schwankungen durch fehlerhafte Einzelbilder und sorgt für visuelle Stabilität der Echtzeitvorhersagen.

6.4 Live-Erkennungsmodul

Das Live-Erkennungsmodul (`camera.html`) bietet dem Benutzer einen freien Nutzungsmodus des Systems ohne spezifisches Ziel. Der Benutzer sieht in Echtzeit den Videoframe mit annotierter *Bounding Box* der erkannten Hand, dem erkannten ASL-Buchstaben und dem entsprechenden *Confidence-Score*.

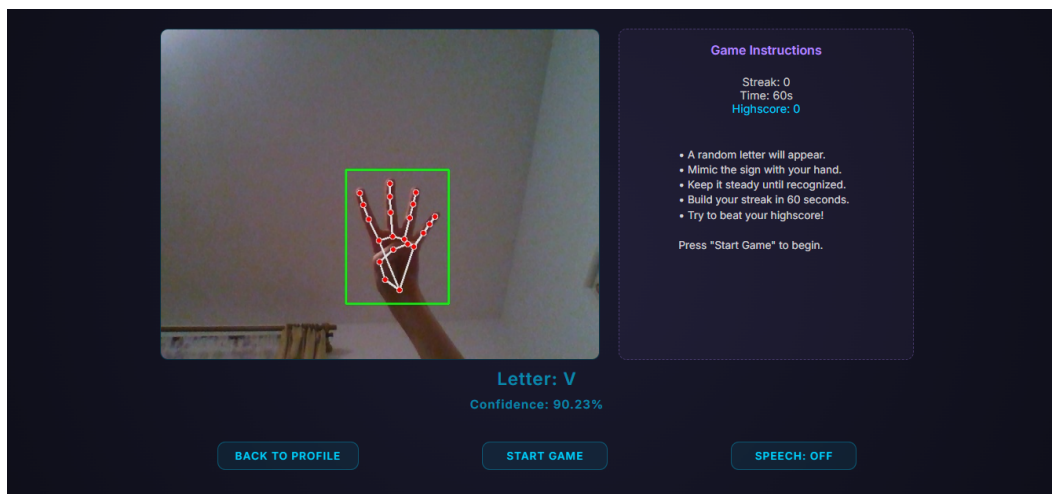


Abbildung 6.2: Live-Erkennungsmodul mit Echtzeitvorhersage

Dieses Modul enthält auch einen **Spielmodus** (*game mode*): Dem Benutzer wird ein zufälliger Buchstabe aus dem ASL-Alphabet angezeigt, und er hat 60 Sekunden Zeit, so viele korrekte Gesten wie möglich hintereinander auszuführen und einen *Streak* aufzubauen. Eine Vorhersage gilt als korrekt, wenn der durch Majority Voting stabilisierte Buchstabe dem Zielbuchstaben entspricht und der durchschnittliche *Confidence-Score* den Schwellenwert von 80% überschreitet. Am Ende des Spiels wird der erreichte Score über den Endpoint `POST /users/save-score` in Firestore gespeichert, und der historische Höchstscore des Benutzers wird aktualisiert und angezeigt.

Das Modul enthält außerdem eine Funktion zur Sprachsynthese (*text-to-speech*): Der erkannte Buchstabe wird automatisch über die `SpeechSynthesis-API` des Browsers ausgesprochen, optional vom Benutzer aktivierbar.

6.5 Modul zum Erlernen des ASL-Alphabets

Das Lernmodul (`learning.html`, `learn-letter.html`) bietet dem Benutzer einen strukturierten Rahmen zum individuellen Erlernen jedes Buchstabens des ASL-Alphabets. Die Hauptseite des Moduls (`learning.html`) zeigt alle 24 statischen Buchstaben und markiert visuell diejenigen, die der aktuelle Benutzer bereits gelernt hat.

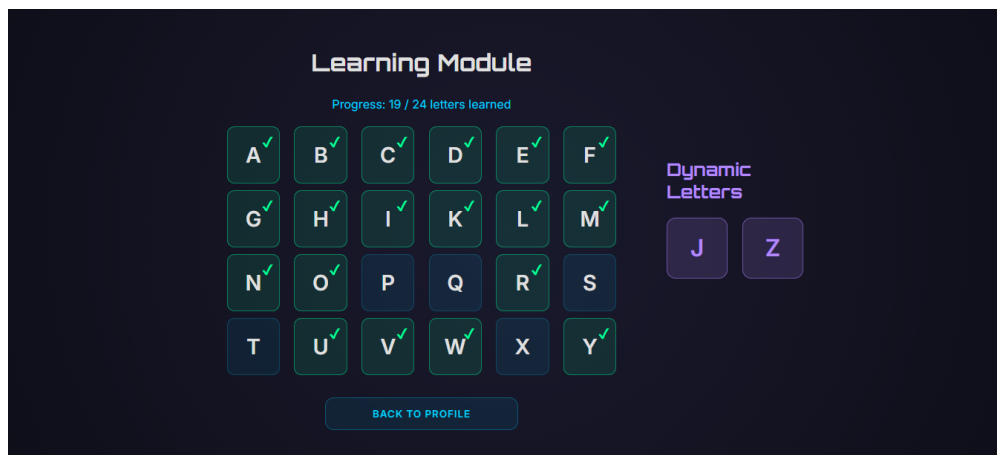


Abbildung 6.3: Lernmodul zum Erlernen des ASL-Alphabets

Bei Auswahl eines Buchstabens wird der Benutzer zur entsprechenden Seite (`learn-letter.html`) weitergeleitet, auf der Folgendes verfügbar ist:

- ein **Referenzbild** der ASL-Geste des ausgewählten Buchstabens als visuelle Orientierung;

- eine **aktive Webcam** mit Echtzeit-Erkennungspipeline, die den annotierten Frame anzeigt und in Echtzeit angibt, ob die ausgeführte Geste korrekt ist oder angepasst werden muss;
- ein **Bestätigungsbutton**, der den Buchstaben als gelernt markiert, ausschließlich verfügbar, wenn die aktuelle Vorhersage korrekt ist und der *Confidence*-Score den Schwellenwert von 80% überschreitet.

Die Logik der Vorhersagestabilisierung ist identisch mit der des Live-Erkennungsmoduls: Majority Voting über die letzten 7 Frames mit einem *Confidence*-Schwellenwert von 80%. Beim Drücken des Bestätigungsbuttons wird der Buchstabe im Benutzerprofil in Firestore über den Endpoint `POST /users/recognize-letter` als gelernt gespeichert, und der Buchstabe wird auf der Hauptseite des Moduls visuell als abgeschlossen markiert.

6.6 Profilseite

Die Profilseite (`profile.html`) stellt die Hauptseite der Anwendung nach der Authentifizierung dar und dient als zentrale Navigationsschnittstelle. Der Benutzer kann seine Kontoinformationen einsehen und zu den beiden Hauptmodulen navigieren: dem Live-Erkennungsmodul und dem Modul zum Erlernen des ASL-Alphabets. Außerdem zeigt die Seite den maximalen Score des Benutzers an, synchronisiert aus Firestore.

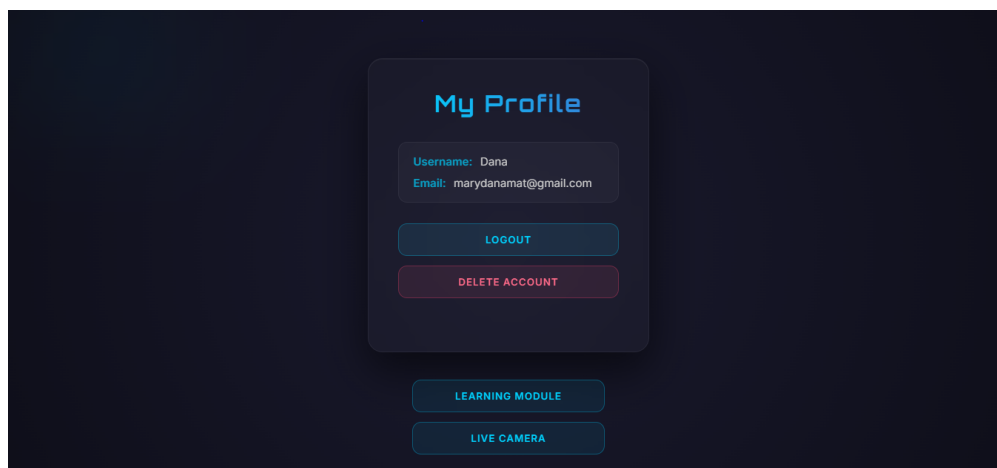


Abbildung 6.4: Profilseite der Anwendung

Kapitel 7

Schlussfolgerungen

7.1 Zusammenfassung der Arbeit

Das Hauptziel dieser Arbeit war die Entwicklung eines Systems zur automatischen Erkennung der statischen Buchstaben des ASL-Alphabets aus Echtzeitbildern, die über eine Webcam aufgenommen werden, basierend auf Convolutional Neural Networks. Dieses Ziel wurde durch die Durchführung aller Phasen eines vollständigen Machine-Learning-Projekts erreicht: Aufbau des Datensatzes, Training und Evaluierung der Modelle sowie Integration des ausgewählten Modells in eine funktionsfähige Desktop-Anwendung.

Im Rahmen dieser Arbeit wurden zwei CNN-Modelle zur Klassifikation der 24 statischen Buchstaben des ASL-Alphabets entwickelt und verglichen. Das erste Modell, ein von Grund auf trainiertes CNN, diente als Baseline und erreichte eine Genauigkeit von 98,17% auf dem Testdatensatz. Das zweite Modell, basierend auf der MobileNetV2-Architektur, die durch Transfer Learning und zweiphasiges Fine-Tuning angepasst wurde, erzielte eine Genauigkeit von 99,59% und übertraf damit das Baseline-Modell um 1,42 Prozentpunkte, wobei sämtliche Klassen mit einer Leistung unter 97% eliminiert wurden. Das MobileNetV2-Modell wurde anschließend in die Desktop-Anwendung *ASL Recognition System* integriert.

7.2 Eigene Beiträge

Die originalen Beiträge dieser Arbeit sind:

- **Aufbau eines kombinierten Datensatzes** mit 103.379 ASL-Bildern, erstellt durch Aggregation, manuelle Bereinigung und Standardisierung von sieben öffentlichen Datensätzen von Kaggle, einschließlich der Spiegelung von Bildern der linken Hand zur Vereinheitlichung auf die rechte-Hand-Konvention;

- **Entwurf, Training und vergleichende Evaluierung** von zwei CNN-Architekturen auf dem erstellten Datensatz, wobei die Überlegenheit von Transfer Learning gegenüber dem Training von Grund auf sowohl hinsichtlich Genauigkeit als auch parametrischer Effizienz gezeigt wurde;
- **Entwicklung der Desktop-Anwendung *ASL Recognition System***, die eine vollständige Echtzeit-Erkennungspipeline (MediaPipe Hands + MobileNetV2 + OpenCV), Benutzer-Authentifizierung über Firebase, ein interaktives Lernmodul für das ASL-Alphabet sowie einen Spielmodus mit Punktesystem integriert.

7.3 Einschränkungen des Systems

Trotz der sehr guten Ergebnisse auf dem Testdatensatz weist das System mehrere Einschränkungen auf, die bei der praktischen Nutzung berücksichtigt werden müssen.

Abhängigkeit von Licht- und Hintergrundbedingungen. Obwohl der Datensatz aus vielfältigen Quellen aufgebaut wurde, kann die Leistung des Systems in Echtzeitszenarien bei sehr schlechter Beleuchtung oder unruhigen Hintergründen abnehmen, da diese die Qualität der Handdetektion durch MediaPipe Hands beeinflussen.

Ausschließlich statische Gesten. Das System erkennt ausschließlich die 24 statischen Buchstaben des ASL-Alphabets, wobei die Buchstaben J und Z, die Bewegungen erfordern, ausgeschlossen sind. Eine Erweiterung auf dynamische Gesten würde einen anderen Ansatz erfordern, der auf der Analyse zeitlicher Bildsequenzen basiert.

Einzelne Hand im Fokus. Die Erkennungspipeline ist für die Detektion einer einzelnen Hand im Bild optimiert. Mehrere Hände oder Objekte, die fälschlicherweise als Hand erkannt werden, können die Vorhersagen beeinträchtigen.

Variabilität zwischen Nutzern. Obwohl der Datensatz mehrere Nutzer abdeckt, kann die Leistung je nach Handform, Ausführungswinkel und individuellem Stil variieren.

Manueller Start der Anwendung. Die Anwendung erfordert das manuelle Starten des Backend-Servers über das Terminal, was die Zugänglichkeit für technisch weniger versierte Nutzer einschränkt.

7.4 Zukünftige Entwicklungsrichtungen

Basierend auf den identifizierten Einschränkungen und den erzielten Ergebnissen werden folgende zukünftige Entwicklungsrichtungen vorgeschlagen:

- **Erkennung dynamischer Gesten.** Erweiterung des Systems zur Erkennung der Buchstaben J und Z durch Integration von Architekturen zur Verarbeitung zeitlicher Sequenzen, wie rekurrente neuronale Netze (LSTM) oder aufmerksamkeitbasierte Modelle;
- **Erkennung von ASL-Wörtern und -Sätzen.** Erweiterung über das Fingeralphabet hinaus durch Training eines Modells zur Erkennung kompletter ASL-Zeichen statt einzelner Buchstaben;
- **Verbesserung der Robustheit in realen Bedingungen.** Sammlung zusätzlicher Daten unter variierenden Licht- und Hintergrundbedingungen oder Anwendung fortgeschrittener Augmentierungstechniken zur Erhöhung der Robustheit gegenüber realen Umgebungen;
- **Packaging und Distribution.** Verpackung der Anwendung als Standalone-Executable, wodurch der manuelle Start des Servers entfällt und die Anwendung einem breiteren Publikum zugänglich gemacht wird;
- **Erweiterung des Lernmoduls.** Hinzufügen detaillierter Nutzerstatistiken, Fortschrittsverfolgung und Übungen zur Wortbildung aus einzelnen Buchstaben, um die Anwendung zu einem umfassenderen Lernwerkzeug auszubauen.

Literaturverzeichnis

- [1] P. K. Pisharady and M. Saerbeck. Real-time hand gesture recognition using fine-tuned convolutional neural network. *Sensors*, 22(1):1–18, 2022.
- [2] G. Benitez-Garcia, J. Prado-Alvarado, and J. Sanchez-Anciga. Visual static hand gesture recognition using convolutional neural network. *Algorithms*, 16(8):361–375, 2023.
- [3] S. Tiwari. Computer vision based hand gesture recognition for american sign language: A review. *Artificial Intelligence Review*, 54(8):6307–6341, 2021.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen. MobileNetV2: Inverted residuals and linear bottlenecks. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 34:4510–4520, 2018.
- [5] J. Deng, W. Dong, R. Socher, L. J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 21:248–255, 2009.
- [6] F. Zhang, V. Bazavan, and A. Vakunov. Mediapipe hands: On-device real-time hand tracking. arXiv preprint arXiv:2006.10214, 2020.
- [7] R. E. Mitchell, T. A. Young, B. Bachleda, and M. A. Karchmer. How many people use ASL in the United States? Why estimates need updating. *Sign Language Studies*, 6(3):306–335, 2006.
- [8] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, USA, 2016.
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

- [11] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning (ICML)*, 37:448–456, 2015.
- [12] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 1:1–15, 2014.
- [13] C. Shorten and T. M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019.
- [14] S. J. Pan and Q. Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [15] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.
- [16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 28:770–778, 2016.
- [17] C. Lugaresi, J. Veach, J. Wang, and G. Kuehne. Mediapipe: A framework for building perception pipelines. *CVPR Workshops*, 3:14–22, 2019.
- [18] M. Taskiran, M. Killioglu, and N. Kahraman. Hand gesture recognition using CNN. *IEEE Conference Publication*, 12:45–53, 2021.
- [19] D. Sau. ASL american sign language alphabet dataset. <https://www.kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset>, 2023.
- [20] K. Londhe. American sign language dataset. <https://www.kaggle.com/datasets/kapillondhe/american-sign-language>, 2022.
- [21] K. Akash. ASL alphabet. <https://www.kaggle.com/datasets/grassknotted/asl-alphabet>, 2018.
- [22] D. Rasband. ASL alphabet test. <https://www.kaggle.com/datasets/danrasband/asl-alphabet-test>, 2019.
- [23] M. Geislinger. ASL RGB-depth fingerspelling: Spelling it out. <https://www.kaggle.com/datasets/mrgeislinger/asl-rgb-depth-fingerspelling-spelling-it-out>, 2019.
- [24] K. U. Kwashe. Significant ASL sign language alphabet dataset. <https://www.kaggle.com/datasets/kuzivakwashe/significant-asl-sign-language-alphabet-dataset>, 2022.

- [25] S. Godhani. ASL hand alphabet dataset. <https://www.kaggle.com/datasets/shreyagodhani645/asl-hand-alphabet-dataset>, 2023.
- [26] J. Singha and K. Das. Hand gesture recognition based on digital image processing. *International Conference on Advanced Computational Intelligence (ICACI)*, 9:253–258, 2018.
- [27] O. Russakovsky, J. Deng, H. Su, and J. Krause. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.
- [28] P. Micikevicius, S. Narang, and J. Alben. Mixed precision training. *International Conference on Learning Representations (ICLR)*, 6:1–12, 2018.
- [29] S. Ramírez. Fastapi framework, high performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>, 2020.
- [30] Google Firebase. Firebase authentication documentation and security guidelines. <https://firebase.google.com/docs/auth>, 2023.
- [31] T. W. Fields. Majority voting and its application in robust decision making systems. *Information Sciences*, 84(3):189–204, 1995.